



Error Handling of In-vehicle Networks Makes Them Vulnerable

**Cyber-Physical System and IOT
University of Padua**

**Biddut Bhowmik
2141802**

Table of Contents

1. Objectives	2
2. System Setup	2
3. Experiments	2
3.1. Packet Class	3
3.2. ECU Class	4
3.3. CANBus Class	4
3.4. Simulation Function	5
4. Results and Discussion	5
5. Conclusion	7

1. Objectives

This project implements and analyzes a simulation based on the reference paper “Error Handling of In-Vehicle Networks Makes Them Vulnerable.” The paper discusses a vulnerability in in-vehicle CAN networks that can be exploited to trigger a bus-off attack on an Electronic Control Unit (ECU), potentially disabling key components in a vehicle.

Project goals:

- Build a simplified simulator of the CAN bus and ECUs capable of sending messages over the bus.
- Simulate and visualize the effects of a bus-off attack.

2. System Setup

- Python version: 3.12.8 (but Python 3.11+ is suitable)
- Libraries:
 - time
 - threading
 - matplotlib.pyplot

How to run the simulator:

- In the terminal:

```
python3 simulator_name.py
```

or,

```
python simulator_name.py
```

- If needed, set up a virtual environment:

```
python -m venv /path/to/new/virtual/environment  
source /path/to/new/virtual/environment/bin/activate
```

3. Experiments

The simulator is implemented using three primary classes:

- **Packet:** Represents the packets transmitted over the CAN bus.

- **ECU (Electronic Control Unit):** Simulates both normal and adversarial behaviors, maintaining error states and handling retransmissions.
- **CANBus:** Simulates the bus connecting ECUs, managing connections and message logs.

Simulator Creation

The simulator initializes a CAN bus, a victim ECU, and an adversarial ECU. Both ECUs are connected to the bus. Two threads are started to simulate the concurrent operation of the ECUs, and their interaction is managed by a threading barrier. The TEC (Transmit Error Counter) history is plotted at the end.

Example code:

```

▶ canbus = CANBus()
  ecu1 = ECU(1)
  ecu2 = ECU(2, attack_mode=ADVERSARY_ECU, adv_target=1)
  # Connect the ECUs to the CAN bus
  canbus.connect_ecu(ecu1)
  canbus.connect_ecu(ecu2)
  # Create a barrier for synchronizing two thread
  barrier = threading.Barrier(2)
  # Start two thread for the two ECUs
  thread1 = threading.Thread(target=run_ecu, args=(ecu1, 100, [1, 0, 1], canbus, barrier))
  thread2 = threading.Thread(target=run_ecu, args=(ecu2, 100, [0, 0, 1], canbus, barrier))

  thread1.start()
  thread2.start()

  thread1.join()
  thread2.join()
  # Plot the final TEC history graph
  plot_tec_history(ecu1, ecu2)

```

Figure 1: Creation of the simulator's part

3.1. Packet Class

The Packet class represents a CAN bus packet with fields for Start/End of Frame, ID, RTR, Control, Data, CRC, and ACK. Its primary function is to provide an array representation for easy comparison when checking for duplicate packets on the bus.

3.2. ECU Class

The ECU class simulates the logic of a real-world ECU. It maintains TEC and REC (Receive Error Counter), manages status changes (Error Active, Error Passive, Bus-Off), and handles retransmission needs.

Key methods include:

- **ecu_status_check()**: Updates the ECU's status based on TEC/REC values.
- **send_packet()**: Sends or retransmits packets over the CAN bus.
- **check_duplicate_ids()**: Looks for duplicate packet IDs and checks for bit mismatches (dominant vs. recessive), triggering retransmissions and error counter increments.
- **tec_rec_check()**: Handles TEC and REC updates based on packet comparison results.

3.3. CANBus Class

The CANBus class simulates the network, keeping track of connected ECUs and packets sent in each round. It includes methods to connect and disconnect ECUs.

```
class CANBus:
    def __init__(self):
        # List of connected ECUs
        self.ecus = []
        # Log of all packets on the bus (it will be cleared at each interaction)
        self.packet_log = []
        # Lock for thread-safe access to the log
        self.lock = threading.Lock()

    def connect_ecu(self, ecu):
        """
        Connect an ECU to the CAN bus.
        An ECU in order to send packets has to be connected to the CAN bus
        """
        if ecu not in self.ecus:
            self.ecus.append(ecu)
            print(f"The ECU {ecu.ecu_id} has been connected to the CAN bus")
        else:
            print(f"The ECU {ecu.ecu_id} is already connected")

    def disconnect_ecu(self, ecu):
        """
        Disconnect an ECU from the CAN bus.
        """
        if ecu in self.ecus:
            self.ecus.remove(ecu)
            print(f"The ECU {ecu.ecu_id} has been disconnected from the CAN bus")
        else:
            print(f"The ECU {ecu.ecu_id} is not connected")
```

Figure 2: CanBus class

3.4. Simulation Function

The main simulation is handled by `run_ecu()`, which:

- Synchronizes ECUs with barriers,
- Sends packets and checks for errors,
- Updates error counters,
- Handles state transitions,
- including disconnecting an ECU in the bus-off state.

4. Results and Discussion

The simulation output is a **TEC history plot** for both ECUs. Typical output includes lines like:

```
The ECU 1 has been connected to the CAN bus
The ECU 2 has been connected to the CAN bus
| [Adversary] | ECU 2 sent packet: ID=100 PACKET=[1, 100, 1, 0, [0, 0, 1], 0, 0, 1] |
| [Normal   ] | ECU 1 sent packet: ID=100 PACKET=[1, 100, 1, 1, [1, 0, 1], 0, 0, 1] |
Adversary -> tec: 8 rec: 8 activeFlag: True passiveFlag: False ErrorActive
Normal    -> tec: 8 rec: 8 activeFlag: True passiveFlag: False ErrorActive
-----
Retrasmission | | [Adversary] | ECU 2 sent packet: ID=100 PACKET=[1, 100, 1, 0, [0, 0, 1], 0, 0, 1] |
Retrasmission | | [Normal   ] | ECU 1 sent packet: ID=100 PACKET=[1, 100, 1, 1, [1, 0, 1], 0, 0, 1] |
Adversary -> tec: 16 rec: 16 activeFlag: True passiveFlag: False ErrorActive
Normal    -> tec: 16 rec: 16 activeFlag: True passiveFlag: False ErrorActive
-----
```

Figure 3: Simulator's output

- The adversary ECU sends a dominant bit-mismatched packet against the victim, incrementing both TEC counters by 8 each round.
- When both ECUs reach a TEC of 128, they transition to Error Passive mode. The adversary's TEC stops increasing, while the victim's continues (now at +7 per round) until it hits 255, entering bus-off and disconnecting from the bus.

TEC Evolution Plot:

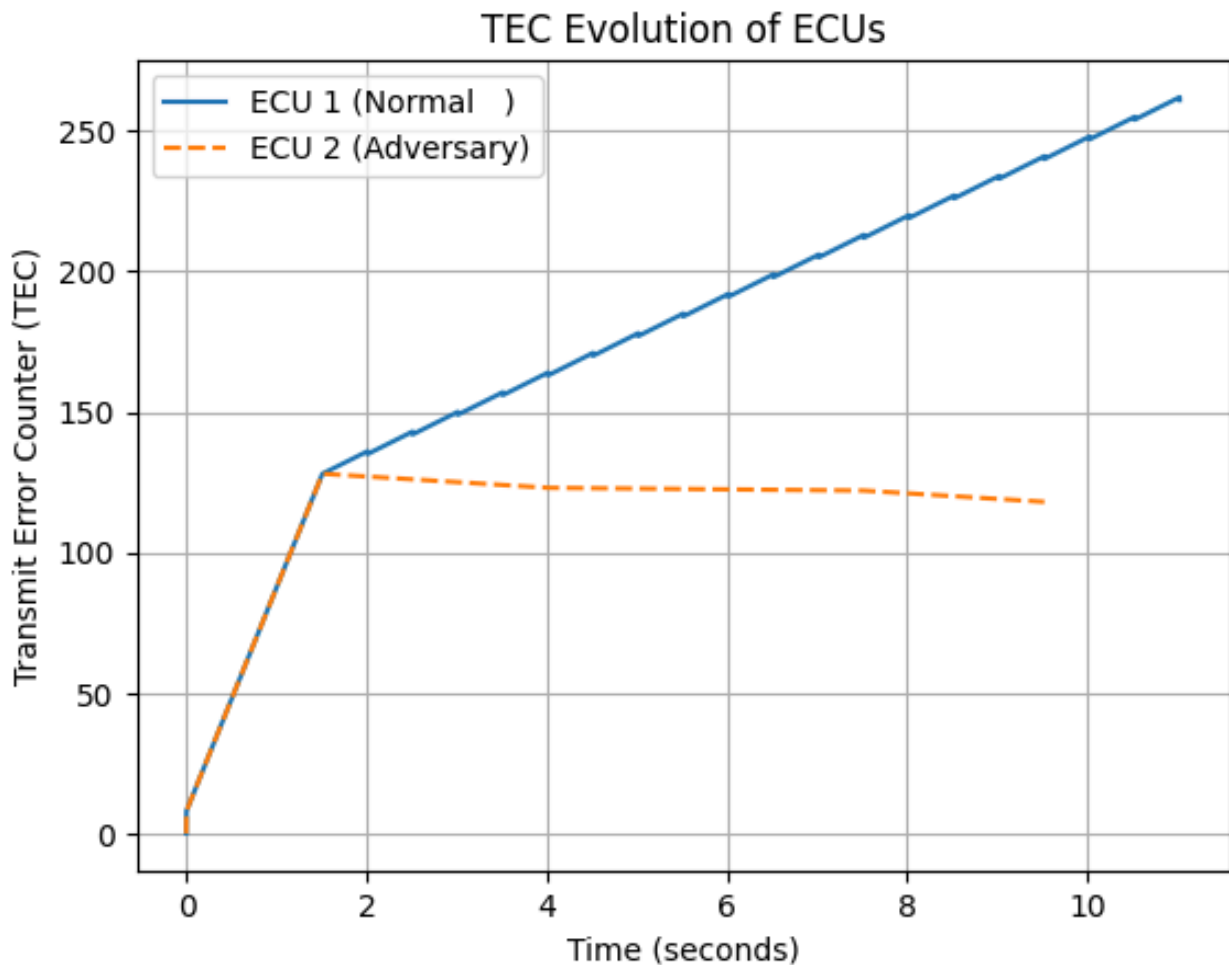


Figure 4: TEC history after the bus-off attack

Analysis:

- The TEC for both ECUs increases quickly at first (by 8 per round), then the victim's TEC continues to rise more slowly in phase 2 (Error Passive), matching observations in the reference paper.
- The simulation's TEC curve matches the expected stepwise rise, though some differences exist due to the simplified model:
 - The adversary attacks immediately (no listening phase).
 - Retransmission delays and flag setting are implemented at a high level, not bit-accurate.

Limitations:

- The simulation does not implement the listening phase for the adversary before attacking.
- All actions are handled at the packet level, not bitwise.
- Threading can lead to minor inconsistencies in flag-setting order, sometimes causing slight deviations in TEC increments.
- Real-world error flags and bitwise arbitration are not modeled.

5. Conclusion

This simulation demonstrates the feasibility and risk of CAN bus-off attacks due to exploitable error handling in the protocol. The results qualitatively match reference findings, illustrating how an attacker can force a critical ECU off the bus.

Future improvements could include bit-level modeling, adversary learning phases, and more advanced error flag handling.

References:

- “Error Handling of In-Vehicle Networks Makes Them Vulnerable,” reference paper.
- GitHub link ([project1](#))